



# *INDIA.RUNS*

Build what next India runs on

Team Name :TAG Star

Team Leader Name :Jash Bhalakia

Problem Statement :Data & AI Challenge – Intelligent Candidate Discovery: rank 100,000 candidates against a Senior AI Engineer JD, fully offline, CPU-only, explainable, and immune to engineered traps.

## Solution Overview

- What is your proposed solution?
- What differentiates your approach from traditional candidate matching systems?

A rule-based + lexical-concept ranking engine that reads a candidate's full career history – not just current title or skills list – and scores fit against a curated AI/ML concept vocabulary, layered with structural trap detection calibrated directly against the real candidate pool.

- Reads career history, not keyword density – a buzzword-stuffed "Marketing Manager" is structurally blocked regardless of how perfect their skills list looks
- Rewards the plain-language builder – a candidate who built a recommendation system but never wrote "RAG" scores correctly, per the JD's own definition of an ideal fit
- Two independent trap-defense layers (honeypot detection + hard disqualifiers), both calibrated by mining the real 100K-candidate pool, not guessed from the brief

## JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?
- Key requirements: embeddings/retrieval, vector DB & hybrid search, ranking/recsys, LLM systems (RAG, fine-tuning), eval frameworks, 5-9y experience (sweet spot 6-8y), Pune/Noida preferred, sub-30-day notice preferred.
- Signals beyond the obvious: career-history depth & recency, company realness (product vs. consulting-only), behavioral signals (recruiter responsiveness, activity recency), skill credibility (proficiency + duration, not just presence).
- Beyond keyword matching: the JD's own test case – "A Tier 5 candidate may not use the words RAG or Pinecone, but if their career history shows they built a recommendation system at a product company, they're a fit." Our semantic-fit score reads full career\_history descriptions, not just title/headline, specifically so this candidate scores correctly.

## Ranking Methodology

- How does your system retrieve, score, and rank candidates?
- What models, algorithms, or heuristics are used?
- How are multiple candidate signals combined into a final ranking?
- Five weighted components: Semantic Fit (55%), Experience Fit (25%, trapezoidal curve peaking 6-8y), Location Fit (12%), Notice-Period Fit (8%) — combined with a multiplicative behavioral modifier (recency, responsiveness, completeness), so inactive candidates are meaningfully down-weighted, not just nudged.
- Why rule-based, not embeddings or an LLM: Section 3 of the spec bans hosted APIs and GPU usage during ranking. A deterministic, lexical + structural approach is the only architecture that's simultaneously compliant, fully explainable, and fast enough to score 100,000 candidates in under 2 minutes on CPU.

## Explainability & Data Validation

- How are ranking decisions explained?
- How do you prevent hallucinations or unsupported justifications?
- How does your solution handle inconsistent, low-quality, or suspicious profiles?
- How decisions are explained: every reasoning string is built from that candidate's own real fields – title, company, years, matched concept buckets, verbatim named skills from their own skills array, location, notice period, any flags. Never a fixed template.
- Hallucination prevention: named skills are pulled directly from the candidate's own data, filtered to advanced/expert proficiency with real duration – a honeypot-style "expert at 0 months" is never cited as credible.
- Suspicious profiles: Honeypots ( $\rightarrow 0.02$ ) – company-founding-year violations on real named companies (e.g. joining Krutrim before 2023) + majority-expert-zero-duration skills. Hard Disqualifiers ( $\rightarrow 0.12$ ) – non-technical titles, consulting-only/research-only careers, AI-hobbyist hedging language.

## End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?
- JD Input → Stream Pool (100K candidates, 5K-row chunks) → Honeypot Check → Disqualifiers → Score & Reason → Sort & Audit (score desc, candidate\_id asc) → Output: `top_100.csv`, validated against `validate_submission.py` before every submission.

## System Architecture

`main.py` (CLI) and `app.py` (Streamlit sandbox) both call → `src/engine.py` (single source of truth for all scoring logic) → Concept Vocabulary / Honeypot Detector / Disqualifier Rules / Scoring Modules / Reasoning Builder → `top_100.csv`.

Zero external dependencies: no requests/urllib/HTTP import anywhere in the ranking path, no torch/transformers/Ollama/hosted embedding API — pure Python stdlib + pandas + numpy. Satisfies Section 3 by construction.

## Results & Performance

- What results or insights demonstrate ranking quality?
- How does your solution meet the challenge's runtime and compute constraints?
- Runtime: <2 minutes on the full 100K pool (budget: 5 min). Honeypots in submitted top 100: 0 (limit  $\leq 10$ ). Hard-disqualified in top 100: 0. Top-20 manually audited: 100% genuine AI/ML titles at real, relevant companies (Amazon, Haptik, Zomato, Niramai, Verloop.io, Yellow.ai, Glance, Mad Street Den, Tech Mahindra, Microsoft).
- Honeypot detector calibrated to 93-111 flagged candidates pool-wide, closely matching the spec's stated "~80 honeypots."



## Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Python 3 (stdlib) — fast scalar date parsing after profiling found `pd.to_datetime()` was ~90% of an earlier version's runtime. pandas + numpy — chunked streaming for constant memory. Streamlit — powers the sandbox demo. openpyxl — formats the ranked-output spreadsheet.

Deliberately excluded: hosted embedding/LLM APIs, GPU inference, Ollama, local transformers — not because they wouldn't work, but because Section 3 bans them during ranking.

## Submission Assets

Github video etc

GitHub Repository:

<https://github.com/bhalakiajash2009-prog/redrob-candidate-ranker>. Ranked

Output: [TAG\\_Star\\_Ranked\\_Candidates.xlsx](#) — top-100 ranked candidates with full reasoning, formatted for review.



# *INDIA.RUNS*

Build what next India runs on

# THANK YOU